BDSRAE PROJECT | TRACK A: LAKEHOUSE AND DATA PLATFORMS | PROJECT 2

Time Travel for Reproducible ML

Detailed project plan: pipeline design, reproducibility test, rollback, storage overhead, retention policy and audit-readiness report

The brief (from your assignment)

Case study. During an audit, a bank must prove exactly which data a credit model was trained on.

What to do. Build a training pipeline on Iceberg that records the snapshot ID with every model version. Demonstrate rollback and exact retraining. Measure the storage overhead of keeping snapshots and propose a retention policy.

Submit. Pipeline code, a reproducibility test that passes, and an audit-readiness report.

Datasets suggested by your dataset sheet: Home Credit Default Risk (Kaggle) and LendingClub loan data. The sheet says to *load in monthly batches to create many snapshots*.

How to read this document. Sections 1-3 explain what is being asked and the key decisions. Sections 4-9 are the build plan (design, code skeletons, tests, experiments). Sections 10-13 cover the report, schedule, risks and a final checklist. Code shown is illustrative skeleton code: I have not run it against a live Spark/Iceberg cluster, so pin versions and test as you build.

Contents

Contents 2

1. Executive summary 3

2. Understanding the assignment 3

2.1 Requirement traceability . . . 3

2.2 Concepts you must be able to explain in the viva / report . . . 4

3. Key decisions (made for you, with reasons) 4

3.1 Dataset: use LendingClub as primary, Home Credit as secondary . . . 4

3.2 Technology stack . . . 4

4. Architecture and data model 6

4.1 Tables . . . 6

4.2 Model registry schema . . . 6

4.3 Table DDL . . . 7

5. Repository layout and module responsibilities 8

5.1 Spark session . . . 8

5.2 Monthly ingest (creates one snapshot per month) . . . 8

5.3 Optional: stamp snapshots with provenance . . . 8

6. Core demonstrations 9

6.1 Training with a pinned snapshot ID . . . 9

6.2 Exact retraining . . . 10

6.3 Rollback . . . 10

7. Reproducibility test suite 11

8. Storage overhead measurement 11

8.1 What to measure . . . 11

8.2 Experiments . . . 12

9. Retention policy proposal 12

10. Audit-readiness report: outline and evidence matrix 14

10.1 Auditor question -> evidence matrix . . . 14

10.2 Responsible-AI notes to include (brief) . . . 14

11. Schedule, milestones and work split 14

11.1 Suggested split for a group of 3-4 . . . 15

12. Risks and mitigations 15

13. Definition of done (final checklist) 15

Appendix A. Command cheat-sheet (Spark SQL) . . . 16

Appendix B. Things to verify against your installed versions . . . 16

1. Executive summary

A bank auditor will ask one question in several forms: "*Show me exactly what data this credit model learned from, and prove you can get the same model back.*" Iceberg helps because every write creates an immutable **snapshot** with a unique ID, and any past snapshot can be queried again. The project is to turn that capability into an auditable ML process.

The plan in one paragraph

Load loan data into one Iceberg table in monthly batches so each month becomes one snapshot. Every training run first **pins** the current snapshot ID, reads the data strictly from that ID, trains with fully deterministic settings, and writes an append-only row into a model registry that binds *model version* ↔ *snapshot ID* ↔ *code commit* ↔ *parameters* ↔ *data fingerprint* ↔ *artifact hash*. A test suite then proves three things: (1) retraining from a recorded snapshot ID reproduces the same data and the same model, even after the table has moved on; (2) a bad batch can be rolled back and the model rebuilt from known-good data; (3) snapshots needed as evidence survive cleanup. Finally you measure how much extra storage snapshots cost under different update patterns and propose a tiered retention policy.

Deliverable (from brief)	What you will produce	Section
Pipeline code	Python/PySpark package `ttml`: ingest, snapshot utilities, train, registry, retrain, rollback, retention, storage metrics, audit report generator	5, 6
Reproducibility test that passes	pytest suite (6 tests) run in CI/one command, including a negative control that proves the test can fail	7
Demonstrate rollback	Scripted demo: poisoned batch -> detect -> rollback -> quarantine model -> rebuild	6.3
Demonstrate exact retraining	Retrain by snapshot ID after table has changed; compare data fingerprint, predictions, artifact hash	6.2
Measure storage overhead	4 experiments (append-only, CoW updates, MoR updates, compaction) with charts	8
Propose retention policy	Three-tier policy with Iceberg properties, tags, and a safety check script	9
Audit-readiness report	Report with evidence matrix, results, limitations and data card	10

Most important idea: a snapshot ID alone is not enough evidence. The model record must also capture *what was done to the data* (filters, feature code, split rule), *how it was trained* (parameters, seed, library versions), and a *content fingerprint* that lets an auditor confirm the data read today is the data read then.

2. Understanding the assignment

2.1 Requirement traceability

Phrase in the brief	Interpretation	How it will be proven
"prove exactly which data"	Given a model version, anyone can retrieve the identical rows used for training	Registry row -> snapshot ID -> `VERSION AS OF` read -> fingerprint equals stored fingerprint
"records the snapshot ID with every model version"	Automatic, not manual, and tamper-resistant	Training code cannot finish without writing the registry row; ID also embedded in artifact sidecar and tag name
"Demonstrate rollback"	Return the live table to a known-good state after a bad write, and handle models trained on bad data	Fingerprint of table after rollback equals fingerprint before bad batch; tainted model marked quarantined
"exact retraining"	Same data + same code + same params gives same model	Prediction vectors identical; model artifact hash identical (or documented equivalence if not)
"storage overhead of keeping snapshots"	Extra bytes consumed by history vs. only the current table	Ratio = total retained bytes / live bytes, measured against number of snapshots and update rate

Phrase in the brief	Interpretation	How it will be proven
"retention policy"	Rules for how long which snapshots are kept	Tiered policy + automated job + guard that refuses to expire referenced snapshots

2.2 Concepts you must be able to explain in the viva / report

- **Snapshot:** immutable state of the table after one commit. Identified by a 64-bit ID, points to a manifest list, which points to manifests, which point to data files.
- **Time travel:** reading a table at a past snapshot ID or timestamp. Works only while that snapshot's metadata and data files still exist.
- **Rollback:** moving the table's current-snapshot pointer back to an earlier snapshot. History is not rewritten; later snapshots remain until expired.
- **Tag / branch:** named references to a snapshot. A tag with a retention period protects its snapshot from `expire_snapshots`.
- **Snapshot expiration:** deletes old snapshot metadata and the data files only they referenced. This is the tension between storage cost and auditability.
- **Copy-on-write vs merge-on-read:** two ways of handling updates/deletes; they create very different storage overhead.
- **Determinism:** identical bytes in, identical bytes out. Needs fixed row order, seeds, library versions and thread settings.

3. Key decisions (made for you, with reasons)

3.1 Dataset: use LendingClub as primary, Home Credit as secondary

Your sheet lists both. The brief wants *monthly batches* to create many snapshots. That favours LendingClub because it has a real origination date, so a month is a natural batch.

Criterion	LendingClub (accepted loans)	Home Credit Default Risk
Real calendar time column	Yes: `issue_d` (month of loan issue)	No. Fields like `DAYS_BIRTH` are relative to application, so batches must be synthetic
Size (verify on download)	About 2.26 million rows, 150 columns (2007-2018 Q4 version commonly on Kaggle)	About 307k rows, 122 columns in application_train, plus related tables
Natural target	`loan_status`: Fully Paid vs Charged Off (drop loans still current/in grace)	`TARGET` (payment difficulty)
Many monthly snapshots	Up to ~140 real months; use 2012-2016 first for 60	Only via synthetic month assignment (e.g. by hashed ID)
Main trap	**Leakage columns** recorded after the loan was issued must be excluded (see below)	Sensitive attributes (e.g. gender) present: exclude from features and document
Role in the project	**Primary.** All experiments	Secondary. Re-run the demo on it to show the pipeline is dataset-agnostic, with the synthetic-batch caveat stated

Leakage warning (LendingClub). Columns such as total payments received, recoveries, last payment date and last credit pull are only known *after* the outcome. Training on them gives unrealistically good models and a bank auditor would reject it. Create an explicit allow-list of origination-time features (loan amount, term, interest rate, grade, income, DTI, employment length, etc.) in a versioned file and include its hash in the registry. Check the actual column list in your download; names vary between Kaggle versions.

Licence and access. Your dataset sheet says to check licence and access terms and to record every dataset in a data card. Both datasets need a free Kaggle account. Confirm the licence on the dataset page at download time and record it in the data card. If you substitute a dataset, explain why in the report (the sheet permits this).

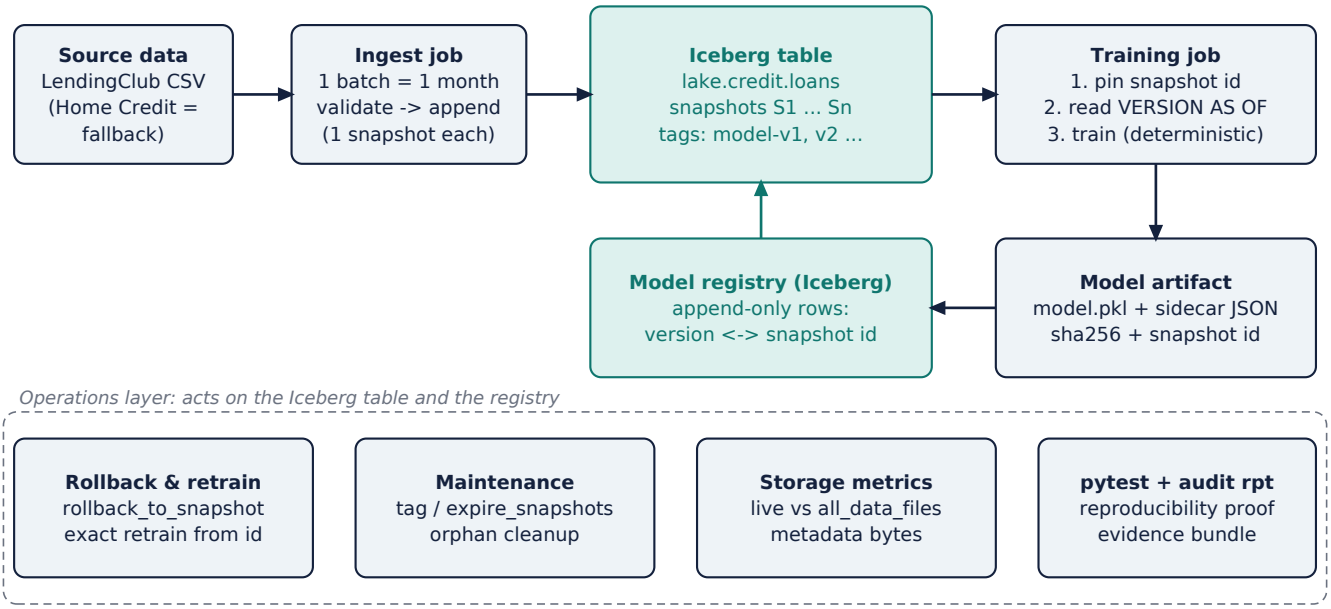
3.2 Technology stack

Layer	Choice	Why / notes
Engine	PySpark 3.5.x with Iceberg Spark runtime for Spark 3.5 / Scala 2.12	Full SQL procedures (rollback, expire, rewrite) and tag syntax. Pin the exact Iceberg version; tag/branch needs Iceberg 1.2 or later
Catalog	Hadoop catalog on local disk (dev); optionally JDBC or REST catalog + MinIO (stretch)	Simplest to run on a laptop. Mention in the report that production uses a shared catalog
Table format	Iceberg format-version 2	Required for row-level deletes (merge-on-read experiments)
Model	LightGBM (fallback: scikit-learn HistGradientBoosting)	Strong on tabular credit data and has deterministic switches
Registry	Iceberg table `lake.ml.model_registry` (source of truth) + optional MLflow mirror	Registry is itself time-travelable, which strengthens the audit story
Tests	pytest with a small fixture table (~20k rows, 6 months)	Fast enough to run in CI in minutes
Environment	Docker image or locked conda/pip environment; Java 11 or 17	Record image digest or lockfile hash in the registry

If your course environment or cluster already provides Spark + Iceberg, use that and skip local setup. The design below does not depend on the catalog type.

4. Architecture and data model

Figure 1. End-to-end architecture



The registry sits between the training job and the Iceberg table: nothing is called a model version unless a registry row exists binding it to a snapshot.

4.1 Tables

Table	Purpose	Key columns / design
`lake.credit.loans`	Training data, one row per loan	loan_id, issue_d, allow-listed features, target, `ingest_batch`. Partition by `months(issue_d)`. Properties: format-version 2, write mode settings per experiment, expiry properties
`lake.ml.model_registry`	Immutable record of every trained model version	See 4.2. Append-only: no UPDATE or DELETE
`lake.ml.model_status_events`	Lifecycle changes (candidate, approved, quarantined, retired)	model_version, status, reason, actor, event_time. Append-only so history is never lost
`lake.ml.ingest_log`	One row per monthly batch load	batch_id, source_file, source_sha256, rows_in, rows_rejected, snapshot_id_after, loaded_at

4.2 Model registry schema

Column	Type	Meaning
model_version	string	e.g. `credit-lgbm-v3` (unique, monotonically increasing)
created_at, trained_by	timestamp, string	When and who/what ran it (user or CI job ID)
table_name	string	Fully qualified Iceberg table
snapshot_id	bigint	**The pinned Iceberg snapshot ID used for all training and evaluation reads**
snapshot_committed_at	timestamp	Commit time of that snapshot (from the snapshots metadata table)
data_filter, split_rule	string	Exact SQL predicate and train/test rule, e.g. issue_d <= 2015-12-31 train, later months test
feature_list_sha256	string	Hash of the versioned feature allow-list file
data_fingerprint	struct(rows bigint, hash string)	Order-independent content hash of the exact training DataFrame (see 6.1)
params_json, seed	string, int	All hyperparameters and seeds

Column	Type	Meaning
git_commit, git_dirty	string, bool	Code version. Refuse to train for a release if dirty
env_lock_sha256, lib_versions	string, map	Dependency lockfile hash and key versions (python, spark, iceberg, lightgbm, numpy)
metrics_json	string	AUC, KS, calibration on the held-out test window
artifact_uri, artifact_sha256	string	Where the model file is and its checksum

4.3 Table DDL

```
CREATE NAMESPACE IF NOT EXISTS lake.credit;
CREATE NAMESPACE IF NOT EXISTS lake.ml;

CREATE TABLE lake.credit.loans (
  loan_id BIGINT, issue_d DATE, loan_amnt DOUBLE, term INT, int_rate DOUBLE,
  grade STRING, annual_inc DOUBLE, dti DOUBLE, emp_length_years DOUBLE,
  -- ... other allow-listed origination-time features ...
  target INT, ingest_batch STRING
) USING iceberg
PARTITIONED BY (months(issue_d))
TBLPROPERTIES (
  'format-version' = '2',
  'write.delete.mode' = 'copy-on-write',
  'write.update.mode' = 'copy-on-write',
  'write.merge.mode'  = 'copy-on-write',
  'history.expire.max-snapshot-age-ms' = '2592000000',   -- 30 days
  'history.expire.min-snapshots-to-keep' = '10'
);
```

5. Repository layout and module responsibilities

```
iceberg-ml-timetravel/
  README.md                 how to run everything in <10 commands
  environment.lock          pinned deps (or Dockerfile with image digest)
  conf/pipeline.yaml        paths, table names, feature allow-list path, seeds
  conf/features_allowlist.txt
  data/raw/                 downloaded CSVs (git-ignored) + data_card.md
  src/ttml/
    spark_session.py        builds SparkSession with Iceberg + catalog
    schema.py               column types, cleaning rules, target definition
    ingest.py               split source into months, validate, append, log
    snapshots.py            current_snapshot_id, list, tag, rollback, fingerprint
    features.py             deterministic feature engineering (pure function)
    train.py                pin snapshot -> read -> train -> save -> register
    registry.py             append rows/events, lookup by version
    retrain.py              retrain(model_version) using registry row only
    rollback.py             rollback + quarantine workflow
    storage_metrics.py      live vs total bytes, metadata bytes
    retention.py            tag, expire, orphan cleanup, safety guard
    audit_report.py         builds evidence bundle (JSON + markdown tables)
  tests/
    conftest.py             small fixture table, temp warehouse
    test_registry_binding.py
    test_exact_retrain.py
    test_rollback.py
    test_retention_safety.py
    test_negative_control.py
  experiments/
    run_storage_experiments.py   produces CSVs + charts for section 8
  scripts/run_demo.sh       end-to-end demo with printed evidence
  reports/                  audit_readiness_report.pdf, figures/
```

5.1 Spark session

```
from pyspark.sql import SparkSession

def get_spark(warehouse="./warehouse", iceberg_version="<PIN_ME>"):
    return (SparkSession.builder.appName("ttml")
      .config("spark.jars.packages",
              f"org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:{iceberg_version}")
      .config("spark.sql.extensions",
              "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions")
      .config("spark.sql.catalog.lake", "org.apache.iceberg.spark.SparkCatalog")
      .config("spark.sql.catalog.lake.type", "hadoop")
      .config("spark.sql.catalog.lake.warehouse", warehouse)
      .config("spark.sql.shuffle.partitions", "8")      # small data
      .getOrCreate())
```

5.2 Monthly ingest (creates one snapshot per month)

Sort the source by `issue_d`, split into calendar months, validate each batch, then append. Every `append()` is one commit, hence one snapshot. Log the resulting snapshot ID.

```
def ingest_month(spark, df_month, batch_id, source_file):
    checks = validate(df_month)                 # nulls, ranges, duplicate loan_id, label rate
    if not checks.ok:
        raise ValueError(f"batch {batch_id} rejected: {checks.reasons}")
    (df_month.withColumn("ingest_batch", F.lit(batch_id))
             .writeTo("lake.credit.loans").append())
    sid = current_snapshot_id(spark, "lake.credit.loans")
    log_ingest(spark, batch_id, source_file, sha256(source_file),
               rows_in=df_month.count(), snapshot_id_after=sid)
    return sid
```

Why use the `refs` table for the current ID? Querying it for the `main` branch is safer than "latest by commit time" if another writer commits concurrently.

```
def current_snapshot_id(spark, table):
    return spark.sql(f"SELECT snapshot_id FROM {table}.refs WHERE name = 'main'").first()[0]
```

5.3 Optional: stamp snapshots with provenance

Iceberg lets you attach custom key/value properties to a commit's snapshot summary (for example the batch ID or the git commit of the loader) through a write option prefixed `snapshot-property.`. This makes the `snapshots` metadata table itself carry audit context. Verify the option name against the Iceberg version you pin before relying on it, and fall back to the `ingest_log` table if it is unavailable.

6. Core demonstrations

6.1 Training with a pinned snapshot ID

Training has a strict order. The snapshot ID is read **once** at the start, and every later read uses that ID, so a concurrent ingest cannot change the training data midway.

Step	Action	Failure mode it prevents
1	Resolve snapshot ID (or accept one as argument for retraining) and verify it exists	Training on a snapshot that was expired
2	Read with `.option("snapshot-id", sid)`; apply filter + split rule	Reading a moving 'latest' table
3	Select allow-listed features, sort by `loan_id`, compute data fingerprint	Hidden order dependence; unprovable data content
4	Train with deterministic params; evaluate on later months	Random split nondeterminism
5	Save model + sidecar JSON, compute sha256	Unverifiable artifact
6	Append registry row; create tag `model-vN` on the snapshot	Model without lineage; snapshot later expired

```
def train(spark, model_version, snapshot_id=None, cfg=CFG):
    sid = snapshot_id or current_snapshot_id(spark, cfg.table)       # pin ONCE
    df = (spark.read.format("iceberg").option("snapshot-id", sid).load(cfg.table))
    df = df.filter(cfg.data_filter).select(*cfg.features, "loan_id", "issue_d", "target") \
           .orderBy("loan_id")
    fp = fingerprint(df)                                             # rows + hash
    train_df = df.filter(f"issue_d <= DATE '{cfg.train_end}'")      # time-based split
    test_df  = df.filter(f"issue_d >  DATE '{cfg.train_end}'")
    X, y = to_numpy(train_df, cfg.features)
    model = lgb.LGBMClassifier(**cfg.params)                         # see below
    model.fit(X, y)
    metrics = evaluate(model, test_df, cfg.features)
    path = save_model(model, model_version, sid)                     # includes sidecar JSON
    registry.append(model_version=model_version, snapshot_id=sid, data_fingerprint=fp,
                    params=cfg.params, git=git_info(), env=env_info(), metrics=metrics,
                    artifact_uri=path, artifact_sha256=sha256_file(path), ...)
    tag_snapshot(spark, cfg.table, sid, f"model-{model_version}", retain_days=cfg.release_retention_days)
    return model_version
```

Deterministic data fingerprint

An order-independent hash lets you prove two DataFrames contain identical rows without comparing them row by row. Sum a per-row 64-bit hash into a wide decimal so it cannot overflow, and include the row count. Use a fixed, sorted column order.

```
def fingerprint(df):
    cols = sorted(df.columns)
    row_hash = F.xxhash64(*[F.col(c) for c in cols])
    r = df.agg(F.count("*").alias("n"),
               F.sum(row_hash.cast("decimal(38,0)")).alias("h")).first()
    return {"rows": r["n"], "hash": str(r["h"])}
```

Also hash the final numeric feature matrix handed to the model (for example `sha256(X.tobytes())` after sorting by ID). This catches nondeterminism introduced after the Spark read.

Deterministic training settings (checklist)

- Sort rows by a unique key before converting to NumPy. Do not rely on Spark partition order.
- Time-based split by `issue_d` (or a hash of `loan_id`). No `randomSplit` or unseeded shuffles.
- LightGBM: set seed (and the bagging/feature-fraction seeds), `deterministic=True`, `force_row_wise=True` and a fixed `num_threads` (1 is safest for the test).
- Pin Python, NumPy, LightGBM versions via lockfile or image digest. Record them in the registry.
- Run the exact-retrain test on the same machine type/architecture. Floating-point results can differ across CPU architectures or library builds; state this limitation honestly in the report.
- Compare at three levels: **L1** data fingerprint equal, **L2** prediction vectors bit-identical, **L3** serialized model hash equal (strip timestamps/paths from the file before hashing). L1+L2 are mandatory; L3 is the gold standard.

6.2 Exact retraining

Retraining must use **only** what the registry row says, never today's config or today's table state.

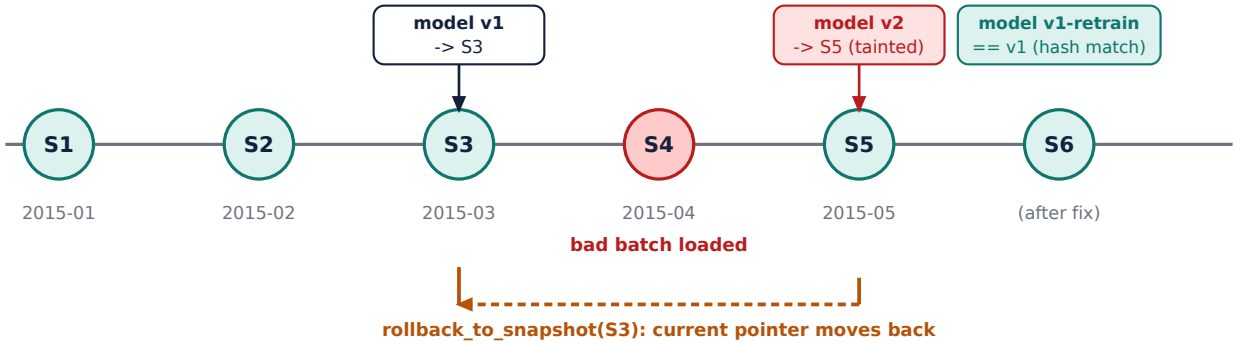
```
def retrain(spark, model_version, new_version):
    row = registry.get(model_version)                 # snapshot_id, params, seed, filter, features...
    assert snapshot_exists(spark, row.table_name, row.snapshot_id), "snapshot expired - evidence lost"
    assert git_commit_available(row.git_commit)       # check out / verify code version
    return train(spark, new_version, snapshot_id=row.snapshot_id,
                 cfg=cfg_from_registry(row))          # same params, features, split
```

Demo script (what the marker should see):

- Ingest months 1-36, train **v1** (snapshot S36). Print the registry row.
- Ingest months 37-48 (table now has 12 newer snapshots). Train **v2** on the head to show the models differ.
- Call `retrain(v1)` -> **v1-retrain**. Print: snapshot IDs equal, data fingerprints equal, predictions identical, artifact hashes equal (or explain exactly which part differs and why).
- Run the same `retrain` after running the retention job, to show the tagged snapshot survived and an untagged old one did not.

6.3 Rollback

Figure 2. Snapshot timeline for the rollback + exact-retrain demo



Tags model-v1 (S3) and model-v2 (S5) stop expire_snapshots from deleting the evidence.

A "poisoned" month (S4) is loaded, a model (v2) is trained on a later head (S5), the problem is found, and the table is rolled back to S3.

Scenario to script: build a corrupted copy of one month (for example, shift a column so `int_rate` holds `loan_amnt`, or flip 30% of labels, or load the same month twice). The ingest validation gate should normally reject this, so for the demo add a flag to *bypass* the gate, then show that a post-hoc data-quality audit catches it.

```
# 1. Detect: compare per-batch stats to history (null rate, mean int_rate, label rate, duplicate ids)
bad = audit_batches(spark, "lake.credit.loans")            # returns the offending ingest_batch / snapshot

# 2. Contain: find models trained on snapshots at or after the bad commit
tainted = registry.models_with_snapshot_in(descendants_of(bad.snapshot_id))
for m in tainted: registry.set_status(m, "quarantined", reason=f"data issue in {bad.batch}")

# 3. Roll back the live table to the last good snapshot (must be an ancestor of current)
spark.sql(f"CALL lake.system.rollback_to_snapshot('credit.loans', {good_snapshot_id})")

# 4. Prove: fingerprint of head == fingerprint recorded when that snapshot was good
assert fingerprint(read_head()) == ingest_fingerprint_at(good_snapshot_id)

# 5. Repair: re-ingest the corrected batch (new snapshot), then retrain a clean model
ingest_month(spark, fixed_df, batch_id, source_file)
retrain_clean_model()
```

Option	SQL / procedure	Use when
Rollback to snapshot	`CALL lake.system.rollback_to_snapshot(table, snapshot_id)`	Target is an ancestor of the current state. The normal choice

Option	SQL / procedure	Use when
Rollback to time	`CALL lake.system.rollback_to_timestamp(table, ts)`	You know when it went wrong but not the ID
Set current snapshot	`CALL lake.system.set_current_snapshot(table, snapshot_id)`	Target is not an ancestor (e.g. after an earlier rollback)
Cherry-pick	`CALL lake.system.cherrypick_snapshot(table, snapshot_id)`	Keep one good staged commit while skipping others

Audit nuance to discuss in the report: rollback changes what is *current*; it does not erase the bad snapshot. The bad snapshot stays inspectable (useful forensics) until expired, and models trained on it must be explicitly quarantined because the table no longer shows the data they used. A rollback itself should be logged (who, when, why) in an append-only table.

7. Reproducibility test suite

The brief requires a test that *passes*. A good suite also shows that the test is capable of failing. Use a small fixture (about 20k rows over 6 months) so the whole suite runs in minutes.

#	Test	Arrange / Act	Assert
T1	registry_binds_snapshot	Train v1	Registry row exists; `snapshot_id` equals the table's main ref at the time; tag `model-v1` points to same ID; sidecar JSON in artifact matches registry
T2	exact_retrain_after_table_moves	Train v1; ingest 2 more months; retrain from v1's registry row	data fingerprint equal; predictions on a fixed probe set identical (`np.array_equal`); artifact hash equal (L3) or documented
T3	time_travel_returns_old_data	Record fingerprint at S3; append; query `VERSION AS OF S3`	Fingerprint equals recorded; row count lower than head
T4	rollback_restores_state	Load good months, record fingerprint F; load poisoned month; rollback	Head fingerprint == F; tainted model status = quarantined; poisoned snapshot still in `snapshots` until expired
T5	retention_preserves_evidence	Tag S3 as model-v1; add many snapshots; run retention	S3 still readable; an old untagged snapshot now raises 'not found'; guard refuses an expire call that would remove a registry-referenced ID
T6	negative_control	Retrain v1's params on the *current head* (no snapshot pin)	Data fingerprint **differs** and predictions differ. Shows T2 is meaningful, not vacuous

```
def test_exact_retrain_after_table_moves(spark, loaded_table, probe_X):
    v1 = train(spark, "v1")
    reg1 = registry.get("v1")
    ingest_more_months(spark, n=2)                      # table head changes
    train.retrain(spark, "v1", "v1-retrain")
    reg2 = registry.get("v1-retrain")

    assert reg2.snapshot_id == reg1.snapshot_id
    assert reg2.data_fingerprint == reg1.data_fingerprint          # L1
    assert np.array_equal(load("v1").predict_proba(probe_X),
                          load("v1-retrain").predict_proba(probe_X))   # L2
    assert reg2.artifact_sha256 == reg1.artifact_sha256            # L3 (normalised file)
```

Run command to put in the README: `pytest -q tests/`. Capture the output in the report as proof. If L3 fails for a benign reason (e.g. embedded timestamp), normalise the file and explain; do not weaken L1 or L2.

8. Storage overhead measurement

8.1 What to measure

Define overhead precisely so numbers are comparable:

Overhead ratio = (total bytes retained on storage) / (bytes of the current table's live data files). A ratio of 1.0 means history is free; 3.0 means history triples the cost. Report data bytes and metadata bytes separately.

```
-- Live (current snapshot) data bytes
```

```
SELECT sum(file_size_in_bytes) AS live_bytes FROM lake.credit.loans.files;

-- All data files reachable from any retained snapshot, counted once each
SELECT sum(file_size_in_bytes) AS retained_data_bytes FROM (
  SELECT DISTINCT file_path, file_size_in_bytes FROM lake.credit.loans.all_data_files);

-- Delete files (merge-on-read) across all snapshots
SELECT sum(file_size_in_bytes) FROM (
  SELECT DISTINCT file_path, file_size_in_bytes FROM lake.credit.loans.all_delete_files);

-- Number of snapshots and manifests
SELECT count(*) FROM lake.credit.loans.snapshots;
SELECT count(*) , sum(length) FROM lake.credit.loans.all_manifests;
```

Cross-check with the filesystem: `du -sb warehouse/credit/loans/data` and `.../metadata`. The difference between the filesystem total and the metadata-table total is orphan files (failed writes), measured separately with `remove_orphan_files` in dry-run mode.

8.2 Experiments

Exp	Workload	Variable	Hypothesis (to test, not assume)
E1	Append-only monthly loads (the base case)	Number of snapshots (12, 24, 48, 96)	Data overhead stays near 1.0 because snapshots share files. Metadata bytes grow slowly with snapshot count
E2	Monthly appends plus corrections via MERGE INTO (e.g. late label updates), **copy-on-write**	Update rate per month: 0.1%, 1%, 5%, 10% of rows	Overhead grows sharply with update rate because each touched file is rewritten and the old version is retained
E3	Same as E2 with **merge-on-read**	Same update rates	Lower write amplification (small delete/position files) but reads need merging. Overhead grows more slowly
E4	Compaction (`rewrite_data_files`) on E1/E3 tables	Before vs. after, with and without expiring the old snapshots	Compaction can **increase** retained bytes until old snapshots are expired, because rewritten files are new files while old files stay reachable
E5 (stretch)	Expiry sweep	Retain last N snapshots (N=1, 5, 10, 30) / older-than window	Plots the cost vs. audit-window trade-off that justifies the retention policy

Outputs per experiment: a CSV (snapshots, live_bytes, retained_data_bytes, delete_bytes, metadata_bytes, ratio) and a chart (x = snapshot count or update rate, y = overhead ratio, one line per mode). Fix the random seed for the synthetic update generator and record it. Because LendingClub has no real update feed, generate corrections by selecting a seeded random sample of existing loans and changing their label or a feature; say so explicitly in the report.

> **Do not present hypotheses as results.** Your report should state what you measured on your data size and cluster, with the caveat that absolute numbers depend on file sizes, partitioning and write settings. Use a dataset large enough that files are not dominated by tiny-file effects (several hundred MB across the experiment is a reasonable aim; scale up per the sheet's "develop small, then scale" note).

9. Retention policy proposal

Principle: **keep what an auditor can ask for; expire what only costs money.** Release evidence is protected by tags; everything else ages out on a schedule.

Tier	Which snapshots	Retention	Mechanism
1. Release evidence	Any snapshot referenced by a registry row whose model was approved or deployed (and quarantined ones, for forensics)	Regulatory period. **Placeholder: 7 years**. Confirm the actual requirement for your jurisdiction and bank policy; do not assume	Tag `model-vN` with `RETAIN n DAYS`; tagged snapshots are excluded from expiry
2. Experiment / candidate	Snapshots used by non-released training runs	90 days	Tag with 90-day retention; auto-expires afterwards

Tier	Which snapshots	Retention	Mechanism
3. Routine ingestion	Snapshots from monthly loads and maintenance	30 days, but always keep the latest 10	Table properties `history.expire.max-snapshot-age-ms` and `min-snapshots-to-keep`
4. Archive (stretch)	Tier-1 snapshots older than 12 months	Same as tier 1 but cheaper storage	Copy the snapshot's data to cold storage with a checksum manifest; keep registry pointer to archive location

```
-- protect a release snapshot (7 years ~ 2555 days; placeholder)
ALTER TABLE lake.credit.loans CREATE TAG `model-v3` AS OF VERSION 8812345678901234567 RETAIN 2555 DAYS;

-- scheduled maintenance, in this order
CALL lake.system.expire_snapshots(table => 'credit.loans',
        older_than => TIMESTAMP '2026-09-27 00:00:00', retain_last => 10);
CALL lake.system.remove_orphan_files(table => 'credit.loans',
        older_than => TIMESTAMP '2026-10-24 00:00:00');       -- keep a safety margin
CALL lake.system.rewrite_manifests('credit.loans');
```

Safety guard (write this; it is a strong marking point)

Before any expiry, the retention job queries the registry for all `snapshot_id` values that are in tiers 1-2. It then checks that each has a live tag or lies within a protected ref. If any is unprotected, the job aborts and reports which model would lose its evidence. This closes the classic failure: a cost-saving cleanup silently destroys audit evidence.

Trade-offs and conflicts to discuss

- **Right to erasure vs. immutability:** time travel can keep personal data that was later deleted. State how your policy handles deletion requests (e.g. expire affected snapshots only after legal review; keep the evidence as aggregated fingerprints; pseudonymise before ingestion).
- **Tamper evidence:** Iceberg snapshot IDs are identifiers, not cryptographic proofs. Someone with storage write access could alter files. Mitigations: object-store versioning or object lock (WORM), storing the data fingerprint in the registry, and signing registry exports.
- **Storage cost vs. audit window:** link directly to experiment E5 and recommend the smallest window that satisfies the audit requirement.
- **Compaction interaction:** after compaction a tagged snapshot still references the old files, so those files are retained; show this effect in E4.

10. Audit-readiness report: outline and evidence matrix

Write this for a reader who is an internal auditor, not a Spark engineer. Aim for 8-12 pages with figures. Suggested structure:

Section	Content	Evidence to include
1. Scope and summary	Model purpose, what was built, headline results, pass/fail on each control	One-page control summary table (below)
2. Data and data card	Dataset source, licence, versions, columns used and excluded (leakage, sensitive), known limitations	Data card (datasheet) per dataset, as your dataset sheet requires; hash of feature allow-list
3. Lineage design	Figure 1 architecture; registry schema; how snapshot IDs are bound to models	Registry sample rows; sidecar JSON example
4. Reproducibility results	Exact retrain across table changes; levels L1/L2/L3 and what they mean	pytest output; fingerprint comparison table; negative control
5. Rollback results	Incident scenario, detection, containment, recovery	Timeline (Figure 2), before/after fingerprints, status events table
6. Storage overhead	Method, experiments E1-E5, charts, interpretation	CSVs, charts, configuration used
7. Retention policy	Tiers, mechanisms, safety guard, legal caveats	Policy table; guard test output
8. Limitations and residual risk	Cross-platform floating point, tamper evidence, synthetic updates, single-node scale	Honest list with mitigations
9. Appendix	How to re-run everything; versions; glossary	README commands; environment lock

10.1 Auditor question -> evidence matrix

Auditor asks	Answer	Where the proof lives
Which data trained model X?	Snapshot ID, filter, feature list, split rule	Registry row; `VERSION AS OF` query output
Is that data still intact?	Re-read the snapshot and compare fingerprint	T3 / T2 output; fingerprint value in registry
Can you rebuild the same model?	Yes, from the registry row only	T2 output with L1/L2/L3 results
What code and settings were used?	Git commit, params, seed, environment lock	Registry columns; lockfile
What if a bad load happened?	Rollback + quarantine procedure, with logs	T4 output; status events table
Could cleanup destroy evidence?	No: tagged snapshots protected; guard aborts	T5 output; retention config
Who did what, and when?	trained_by, created_at, status events, ingest_log	Registry + event tables
What does this cost?	Overhead ratios by workload	Section 8 charts

10.2 Responsible-AI notes to include (brief)

- Document that the credit model is a teaching artefact built on a public dataset, not validated for real lending decisions.
- State which sensitive or proxy attributes were excluded and note that excluding them does not guarantee fairness (the fairness projects in Track D cover that properly).
- Never attempt to re-identify borrowers; treat the data as sensitive even though it is public (per the dataset sheet).

11. Schedule, milestones and work split

The brief gives no deadline, so this is a **5-week template**. Stretch or compress it to your actual submission date but keep the order: build the safety net (tests) before the experiments.

Week	Milestone	Key tasks	Exit criterion
1	Environment + data	Install Spark/Iceberg, lock env. Download dataset, write data card, define feature allow-list and target, build fixture. Create tables	Can append one month and see a snapshot in `.snapshots`
2	Ingest + registry + training	Monthly ingest with validation and ingest_log. Fingerprint function. train(), registry append, tagging	v1 trained; registry row printed with snapshot ID
3	Exact retrain + rollback	retrain(), determinism settings, tests T1-T3, T6. Poisoned batch, rollback workflow, quarantine, T4	All of T1-T4, T6 pass
4	Storage + retention	Storage metrics module, experiments E1-E4 (E5 if time). Retention job, guard, T5	Charts generated from scripts; T5 passes
5	Report + polish	Audit-readiness report, README, full clean-checkout run, demo script, rehearse	Fresh clone runs `scripts/run_demo.sh` and `pytest` green

11.1 Suggested split for a group of 3-4

Workstream	Owns	Hands off to
A. Data and ingest	Data card, cleaning, allow-list, monthly ingest, validation, poisoned-batch generator	B (clean table), C (poisoned data)
B. Training and registry	Training, determinism, fingerprint, registry, retrain, T1-T3, T6	C, D
C. Rollback and retention	Rollback workflow, quarantine, retention job + guard, T4-T5	D
D. Measurement and report	Storage metrics, experiments E1-E5, charts, audit-readiness report, README	Everyone reviews

With 2 people: merge A+B and C+D. Solo: drop E5, the archive tier and the Home Credit rerun.

12. Risks and mitigations

Risk	Likelihood	Mitigation
Retrained model is not bit-identical	Medium	Single thread, deterministic flags, sorted input, same environment; assert L1+L2 strictly, normalise artifact for L3; document residual causes
Leakage columns inflate model and undermine credibility	High if unmanaged	Explicit allow-list, reviewed by a second member; include a leakage section in the data card
Snapshot expired before test/demo	Medium	Tag before running retention; T5 guard; keep a golden-fixture snapshot
Spark/Iceberg version mismatch (jar vs. Spark vs. Scala)	High on first setup	Pin one combination early (week 1); smoke test `.snapshots` and CALL procedures
Storage numbers noisy because data is small	Medium	Scale up data for experiments; repeat runs; report mean and range; separate data from metadata bytes
Rollback semantics misunderstood (history is not erased)	Medium	Show the bad snapshot still listed after rollback; explain in the report
Local Hadoop catalog is not safe for concurrent writers	Low for the demo	Run ingest single-writer; mention JDBC/REST catalog for production
Dataset licence or download changes	Low	Record version and date in the data card; allowed to substitute with justification

13. Definition of done (final checklist)

- ■ Every model version in the registry has a snapshot_id, data fingerprint, git commit, params, seed, environment hash and artifact hash.
- ■ Training cannot complete without writing the registry row and tag.
- ■ T1-T6 pass from a clean checkout; the pytest log is saved in the report.
- ■ Exact retrain shown after the table has changed, with L1 and L2 equal and L3 reported.
- ■ Rollback demo shows: bad snapshot detected, models quarantined, head fingerprint restored, clean model rebuilt.

- ■ Storage experiments E1-E4 run from one script and produce CSVs and charts; overhead ratio defined in the report.
- ■ Retention policy documented with tiers, automated job and safety guard; legal placeholders clearly marked for confirmation.
- ■ Data card(s) completed; licences checked; leakage and sensitive columns documented.
- ■ Audit-readiness report includes the auditor question -> evidence matrix and an honest limitations section.
- ■ README lets a stranger reproduce everything in under 10 commands.

Appendix A. Command cheat-sheet (Spark SQL)

```
-- inspect history
SELECT snapshot_id, committed_at, operation, summary FROM lake.credit.loans.snapshots ORDER BY committed_at;
SELECT * FROM lake.credit.loans.history;
SELECT * FROM lake.credit.loans.refs;                          -- branches and tags
SELECT * FROM lake.credit.loans.manifests;

-- time travel reads
SELECT count(*) FROM lake.credit.loans VERSION AS OF 8812345678901234567;
SELECT count(*) FROM lake.credit.loans VERSION AS OF 'model-v3';           -- by tag
SELECT count(*) FROM lake.credit.loans TIMESTAMP AS OF '2026-10-01 00:00:00';

-- rollback and repair
CALL lake.system.rollback_to_snapshot('credit.loans', 8812345678901234567);
CALL lake.system.set_current_snapshot('credit.loans', 8812345678901234567);

-- maintenance
CALL lake.system.rewrite_data_files(table => 'credit.loans', strategy => 'binpack');
CALL lake.system.expire_snapshots(table => 'credit.loans', retain_last => 10);
CALL lake.system.remove_orphan_files(table => 'credit.loans');
```

Appendix B. Things to verify against your installed versions

- Exact Iceberg and Spark versions, and that `VERSION AS OF` with a tag name and `CREATE TAG` are supported in your combination.
- Spark option names for snapshot reads and for snapshot summary properties.
- Column names and counts in your downloaded LendingClub / Home Credit files (they vary between Kaggle uploads).
- The regulatory retention period for your scenario (the 7-year figure here is a placeholder, not advice).
- Official docs to consult: Apache Iceberg docs (Spark queries, Spark procedures, branching and tagging, maintenance) and the LightGBM parameters page on determinism.